

# NHẬP MÔN HỌC MÁY

*Nền tảng Toán học cho Trí tuệ nhân tạo*

TS. Cao Tiến Dũng



**FPT SCHOOL OF BUSINESS  
& TECHNOLOGY**

# Nền tảng Toán học

- Các hệ thống Trí tuệ nhân tạo (AI) thường dựa vào các phương pháp toán học và thống kê để xử lý dữ liệu, học từ dữ liệu và đưa ra dự đoán.
- **Các lĩnh vực toán cốt lõi:**
  - Đại số tuyến tính (Linear Algebra)
  - Giải tích (Calculus)
  - Lý thuyết xác suất (Probability Theory)
  - Thống kê (Statistics)
  - Tối ưu hóa (Optimization)
- **Ví dụ:** Mạng nơ-ron (Neural Network) dùng Đại số tuyến tính cho phép toán ma trận, Giải tích cho tối ưu dựa trên gradient, và Xác suất cho dự đoán mô hình.

# Vai trò của Toán học trong AI

- Các khái niệm toán học cung cấp một cách tiếp cận có cấu trúc để hiểu các mô hình và thuật toán AI.
- Một số nền tảng toán học tiêu biểu trong AI:
  - Mạng nơ-ron (Neural Network): dùng phép nhân ma trận và đạo hàm cho lan truyền ngược (backpropagation).
  - Máy vector hỗ trợ (SVM): sử dụng tối ưu hóa và hình học.
- Các kỹ thuật toán học giúp AI xử lý những bài toán phức tạp như nhận dạng mẫu, ra quyết định và mô hình hóa dự báo.

01

# Đại số tuyến tính

*Linear Algebra*

# Đại số tuyến tính: Vector và Ma trận

- Vector: mảng số một chiều, dùng để biểu diễn các điểm dữ liệu.
- Ma trận (Matrix): mảng số hai chiều, dùng cho phép biến đổi và biểu diễn dữ liệu.
- Phép toán: cộng vector và ma trận, nhân vô hướng, tích vô hướng (dot product), v.v.
- Ứng dụng: trong AI, ma trận biểu diễn dữ liệu, trọng số (weights) trong mạng nơ-ron, và các phép biến đổi trong thị giác máy tính.

# Ví dụ: Vector & Ma trận trong Python (NumPy)

Biểu diễn các pixel của ảnh dưới dạng ma trận:

```
import numpy as np
# Ảnh xám 3x3 → ma trận
image = np.array([[255, 0, 128],
                  [ 64, 32, 200],
                  [ 10, 90, 170]])
# Mạng nơ-ron: trải ảnh thành vector, nhân với ma trận trọng số
W = np.random.randn(2, 9) # 2 nơ-ron, 9 đầu vào
x = image.flatten()        # shape (9,)
output = W @ x             # nhân ma trận-vector → shape (2,)
```

## Ý tưởng then chốt:

Mỗi chữ số MNIST  $28 \times 28$  = một vector 784 chiều.  $W @ x$  chính là một lớp lan truyền xuôi (forward pass).

# Đại số tuyến tính: Trị riêng & Vector riêng

- **Trị riêng (Eigenvalue) & Vector riêng (Eigenvector):** với ma trận  $A$ , trị riêng  $\lambda$  và vector riêng  $v$  thỏa mãn  $Av = \lambda v$ .
- **Ứng dụng:** giảm chiều dữ liệu (ví dụ Phân tích thành phần chính – PCA) và phân cụm phổ (spectral clustering).
- **Ví dụ:** vector riêng biểu diễn các “hướng” mà dữ liệu trải rộng theo đó, còn trị riêng cho biết “mức độ quan trọng” (phương sai) của các hướng đó.

# Ví dụ: PCA với Trị riêng & Vector riêng

Giảm chiều từ 2D → 1D bằng PCA:

```
from sklearn.decomposition import PCA
import numpy as np
X = np.array([[2.5,2.4],[0.5,0.7],[2.2,2.9],
              [1.9,2.2],[3.1,3.0]])
pca = PCA(n_components=1)
X_reduced = pca.fit_transform(X)
print(pca.explained_variance_ratio_)
# Kết quả: [0.965] → giữ lại 96.5% phương sai chỉ với 1 thành phần
```

## Diễn giải:

- **Trị riêng** = lượng phương sai theo mỗi hướng (hướng đó “quan trọng” đến đâu).
- **Vector riêng** = chính là hướng đó. PCA chỉ giữ lại các hướng quan trọng nhất.

# Đại số tuyến tính: Phân rã giá trị suy biến (SVD)

- SVD: phân rã một ma trận  $A$  thành ba ma trận  $U$ ,  $\Sigma$  và  $V^T$ .
- Trường hợp sử dụng:

Nén dữ liệu (data compression)

Phân tích ngữ nghĩa tiềm ẩn (LSA) trong Xử lý ngôn ngữ tự nhiên

Hệ gợi ý (ví dụ: Netflix dùng SVD để gợi ý phim)

- Ý nghĩa: SVD giúp phát hiện các mẫu trong dữ liệu nhiều chiều bằng cách chiếu nó xuống không gian ít chiều hơn.

# Ví dụ: Nén ảnh bằng SVD

Chỉ giữ lại k giá trị suy biến lớn nhất để nén dữ liệu:

```
import numpy as np
A = np.random.rand(100, 100) # ảnh mô phỏng (100x100)
U, S, Vt = np.linalg.svd(A, full_matrices=False)
k = 10 # giữ 10 giá trị suy biến lớn nhất
A_compressed = U[:, :k] @ np.diag(S[:k]) @ Vt[:k, :]
# Lưu trữ: đầy đủ = 10,000 số
# nén lại = 10*(100+1+100) = 2,010 số (tiết kiệm 80%)
```

## Ứng dụng thực tế:

- Netflix dùng SVD trên ma trận Người dùng × Phim để dự đoán các điểm đánh giá chưa biết.
- LSA trong NLP dùng SVD để tìm các chiều chủ đề ẩn (hidden topics).



02

# Giải tích

*Calculus*

# Giải tích: Đạo hàm và Gradient

- **Đạo hàm (Derivative):** đo lường mức độ thay đổi của một hàm số khi đầu vào thay đổi.
- **Gradient:** mở rộng khái niệm đạo hàm cho hàm nhiều biến.
- **Ứng dụng:** trong học máy, đạo hàm giúp cực tiểu hóa hàm chi phí (cost function) bằng cách điều chỉnh trọng số.
- **Ví dụ:** trong lan truyền ngược (backpropagation) của mạng nơ-ron, gradient được tính để cập nhật trọng số của mạng.

$$w := w - \eta \frac{\partial L}{\partial w}$$

Quy tắc cập nhật gradient descent

# Ví dụ: Gradient Descent từng bước

Hàm mất mát:  $L(w) = (2w - 4)^2$ , cực tiểu tại  $w^* = 2$

Đạo hàm:  $dL/dw = 4(2w - 4)$

```
# Một bước cập nhật (learning rate lr = 0.1):  
w = 0                                # trọng số ban đầu  
gradient = 4*(2*w - 4) = -16          # tại w=0  
w_new = w - lr * gradient              # quy tắc gradient descent  
        = 0 - 0.1 * (-16) = 1.6      # đã tiến về phía w*=2
```

## Trong Mạng nơ-ron:

- Quy tắc dây chuyền (chain rule) mở rộng cách này cho hàng nghìn trọng số thông qua backpropagation.
- Mỗi trọng số được cập nhật đồng thời:  $w := w - lr \cdot (dL/dw)$ .

# Giải tích: Đạo hàm riêng

- **Đạo hàm riêng (Partial Derivative):** đạo hàm của hàm nhiều biến theo một biến, giữ các biến còn lại cố định.
- **Ý nghĩa:** thiết yếu để hiểu mỗi đặc trưng (feature) trong mô hình ảnh hưởng ra sao đến đầu ra.
- **Ứng dụng:** dùng trong các bài toán tối ưu hóa cho mô hình học máy (ví dụ: gradient descent).

# Ví dụ: Đạo hàm riêng trong Hồi quy tuyến tính

Mô hình với 2 đặc trưng:  $f(w_1, w_2) = (w_1 \cdot x_1 + w_2 \cdot x_2 - y)^2$

```
# Các đạo hàm riêng:  
df/dw1 = 2*(w1*x1 + w2*x2 - y) * x1    # gradient cho w1  
df/dw2 = 2*(w1*x1 + w2*x2 - y) * x2    # gradient cho w2  
# Ví dụ số: x1=2, x2=3, y=10, w1=1, w2=1  
prediction = 1*2 + 1*3 = 5  
error      = 5 - 10 = -5  
df/dw1     = 2*(-5)*2 = -20    # tăng w1 để giảm mất mát  
df/dw2     = 2*(-5)*3 = -30    # tăng w2 nhiều hơn (x2 ảnh hưởng lớn hơn)
```

**Diễn giải:**  $df/dw_1$  cho ta biết “nếu  $w_1$  tăng một lượng rất nhỏ thì mất mát thay đổi  $-20$  lần lượng đó” — nên ta cần tăng  $w_1$ .

# Giải tích: Gradient Descent

- **Gradient Descent:** thuật toán lặp dùng để cực tiểu hóa một hàm bằng cách di chuyển theo hướng ngược với gradient.
- **Các biến thể:**
  - **Batch Gradient Descent:** tính gradient trên toàn bộ tập dữ liệu.
  - **Stochastic GD (SGD):** cập nhật trọng số dựa trên một điểm dữ liệu.
  - **Mini-batch GD:** dung hòa giữa batch và stochastic.
- **Ứng dụng:** tối ưu hàm mất mát trong mạng nơ-ron, hồi quy logistic, v.v.

# Ví dụ: So sánh các biến thể Gradient Descent

Ba biến thể được so sánh:

- **Batch GD** — dùng cả N mẫu mỗi lần cập nhật. Ổn định nhưng chậm.
- **SGD** — dùng 1 mẫu mỗi lần cập nhật. Nhanh nhưng nhiễu.
- **Mini-batch** — dùng 32–256 mẫu. Cân bằng cả hai (mặc định trong thực tế).

```
# Mini-batch GD với PyTorch:
for epoch in range(100):
    for X_batch, y_batch in DataLoader(ds, batch_size=32):
        optimizer.zero_grad()
        loss = criterion(model(X_batch), y_batch)
        loss.backward()      # tính gradient
        optimizer.step()     # cập nhật trọng số
```

**Kinh nghiệm:** batch\_size = 32 hoặc 64 là điểm khởi đầu tốt. Batch lớn hơn chạy nhanh hơn nhưng có thể hội tụ về cực tiểu “nhọn” (kém tổng quát).



03

# Xác suất

*Probability Theory*

# Lý thuyết xác suất: Cơ bản

- **Biến ngẫu nhiên (Random Variable):** biến mà giá trị của nó là kết quả của hiện tượng ngẫu nhiên.
- **Phân phối xác suất (Probability Distribution):** hàm gán xác suất cho các kết quả có thể xảy ra (ví dụ: phân phối Chuẩn, phân phối Nhị thức).
- **Ứng dụng:** dùng trong ra quyết định, dự đoán và phân loại (ví dụ: bộ phân loại Naïve Bayes).

# Ví dụ: Bộ lọc thư rác Naïve Bayes

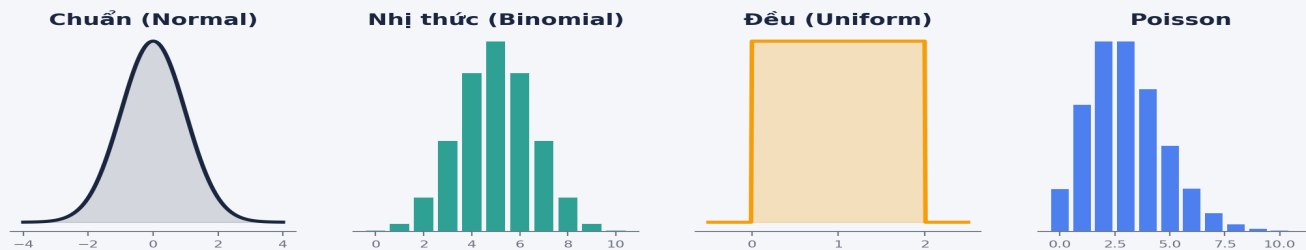
Áp dụng Định lý Bayes để phân loại email:

- **Tiên nghiệm:**  $P(\text{Spam}) = 0.30$  (30% email là thư rác)
- **Khả năng:**  $P(\text{'free'} \mid \text{Spam}) = 0.80$  (80% thư rác chứa từ 'free')
- **Bằng chứng:**  $P(\text{'free'}) = 0.40$  (40% mọi email chứa 'free')

```
# Áp dụng Định lý Bayes:  
P(Spam | 'free') = P('free' | Spam) * P(Spam) / P('free')  
                 = 0.80 * 0.30 / 0.40  
                 = 0.60    → 60% khả năng email này là thư rác  
  
# Cài đặt bằng Python:  
from sklearn.naive_bayes import MultinomialNB  
model = MultinomialNB()  
model.fit(X_train_counts, y_train)    # X = số lần xuất hiện từ  
predictions = model.predict(X_test_counts)
```

# Phân phối xác suất: Rời rạc và Liên tục

- **Phân phối rời rạc (Discrete):** nhận các giá trị cụ thể (ví dụ: phân phối Nhị thức).
- **Phân phối liên tục (Continuous):** nhận giá trị bất kỳ trong một khoảng (ví dụ: phân phối Gauss).
- **Ứng dụng:**
  - **Nhị thức:** dùng trong bài toán phân loại.
  - **Gauss:** thường được giả định trong nhiều mô hình AI (mạng Bayes, mô hình hỗn hợp Gauss).



Một số phân phối xác suất thường gặp

# Ví dụ: Gauss & Nhị thức trong thực tế

Phân phối Gauss — chuẩn hóa điểm thi:

```
import scipy.stats as stats
mean, std = 75, 10
prob_above_90 = 1 - stats.norm.cdf(90, mean, std)
print(f'{prob_above_90:.1%}') # Kết quả: 6.7%
# Chỉ 6.7% sinh viên đạt trên 90 điểm
```

Phân phối Nhị thức — kiểm thử A/B tỷ lệ click:

```
clicks = stats.binom(n=1000, p=0.05)
print(clicks.mean(), clicks.std())
# Kết quả: 50.0 6.89
# Kỳ vọng ~50 click; độ lệch chuẩn 6.89
# cho khoảng tin cậy
```

## Ứng dụng trong AI:

- **Gauss** → chuẩn hóa đặc trưng (feature scaling), mô hình hỗn hợp Gauss, mạng Bayes.
- **Nhị thức** → kiểm thử A/B các phiên bản mô hình, tinh chỉnh ngưỡng phân loại.

# Định lý Bayes

$$P(h \mid d) = \frac{P(d \mid h) P(h)}{P(d)}$$

trong đó:  $d$  = dữ liệu (data),  $h$  = giả thuyết (hypothesis)

- Các thành phần trong quy tắc Bayes:

- **P(h)**: xác suất tiên nghiệm (niềm tin về giả thuyết trước khi thấy dữ liệu).
- **P(d | h)**: khả năng (likelihood) — xác suất quan sát dữ liệu nếu giả thuyết đúng.
- **P(d)**: bằng chứng (evidence) — xác suất biên của dữ liệu.
- **P(h | d)**: hậu nghiệm (posterior) — xác suất của giả thuyết sau khi thấy dữ liệu.

Khai triển bằng chứng:

$$P(d) = \sum_h P(d \mid h) P(h)$$

**Ứng dụng:** suy luận trong mạng Bayes; bộ phân loại Naïve Bayes.



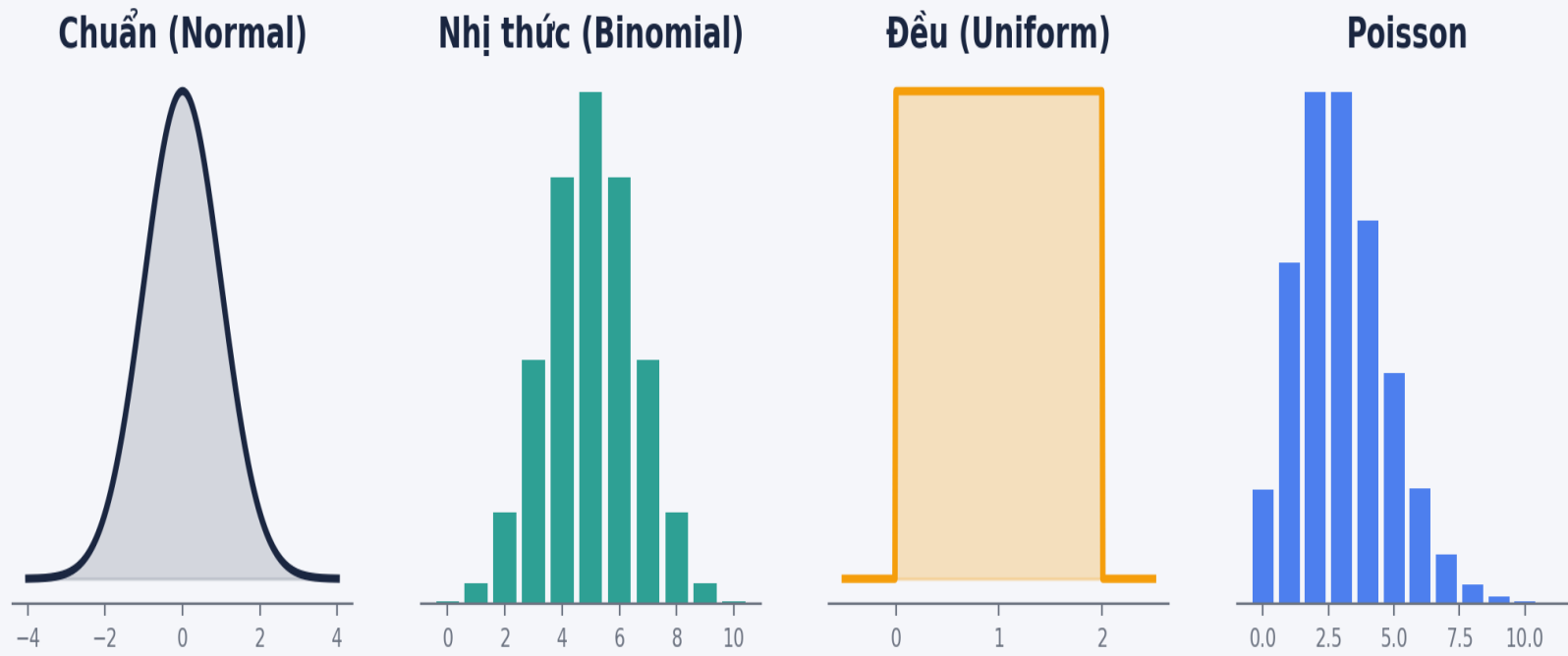
04

# Thống kê

*Statistics*

# Thống kê là gì?

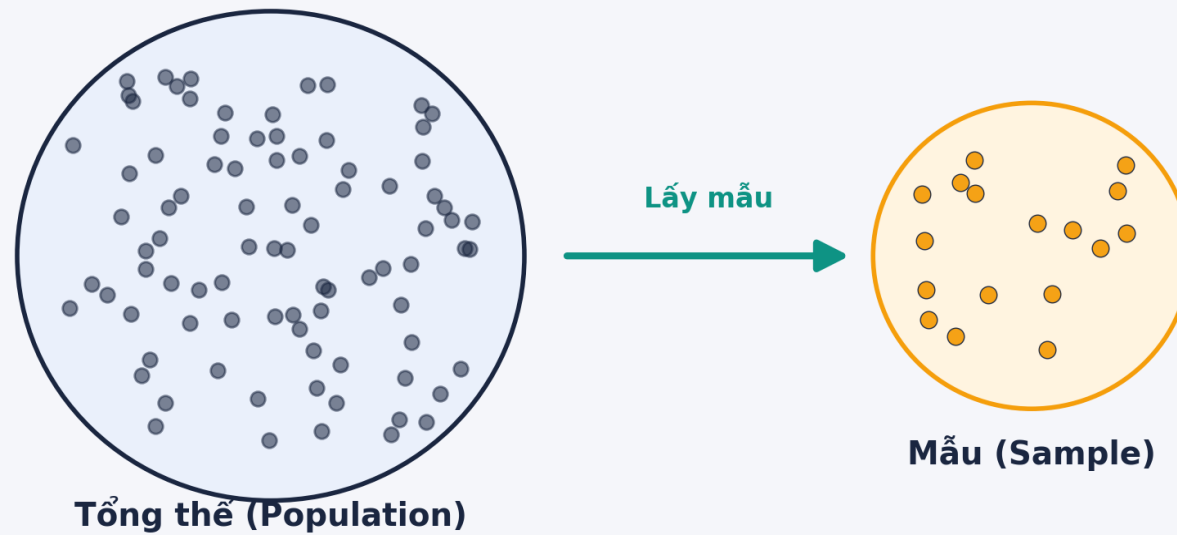
Là một lĩnh vực của toán ứng dụng liên quan đến việc thu thập, phân tích, diễn giải và trình bày dữ liệu.



*Trực quan hóa và mô tả dữ liệu là cốt lõi của thống kê*

# Thuật ngữ thống kê

- **Tổng thể (Population):** toàn bộ nhóm đối tượng mà ta muốn có thông tin về nó.
- **Mẫu (Sample):** một tập con của tổng thể, được lấy ra vì tổng thể thường quá lớn để phân tích trọn vẹn. Đặc trưng của mẫu được xem là đại diện cho tổng thể.



# Ví dụ: Tổng thể vs Mẫu trong nghiên cứu ML

**Bối cảnh:** nghiên cứu kết quả thi của sinh viên CNTT tại Việt Nam.

- **Tổng thể:** tất cả sinh viên CNTT tại Việt Nam (~500.000 SV ở hơn 200 trường) — không thể thu thập từ mọi sinh viên.
- **Mẫu:** 200 sinh viên chọn ngẫu nhiên từ 5 trường. Đo: GPA, số giờ học/tuần, ngôn ngữ lập trình sử dụng.

```
# Suy diễn thống kê:  
sample_mean_gpa = 7.2    # đo từ 200 sinh viên  
margin_of_error = 0.3    # ở mức tin cậy 95%  
# Kết luận: GPA của tổng thể xấp xỉ 7.2 +/- 0.3
```

**Liên hệ với ML:** tập huấn luyện = mẫu. Tập kiểm thử đánh giá mức độ tổng quát hóa của mô hình lên toàn bộ tổng thể (dữ liệu chưa thấy).

# Kỹ thuật lấy mẫu

- **Lấy mẫu xác suất (Probability):**

- **Ngẫu nhiên (Random):** mỗi phần tử có cơ hội được chọn như nhau.
- **Hệ thống (Systematic):** chọn theo bước cố định (ví dụ cứ 2 bản ghi chọn 1).
- **Phân tầng (Stratified):** chia tổng thể thành các nhóm con cùng đặc điểm, rồi chọn ngẫu nhiên trong mỗi nhóm.

- **Lấy mẫu phi xác suất (Non-Probability):**

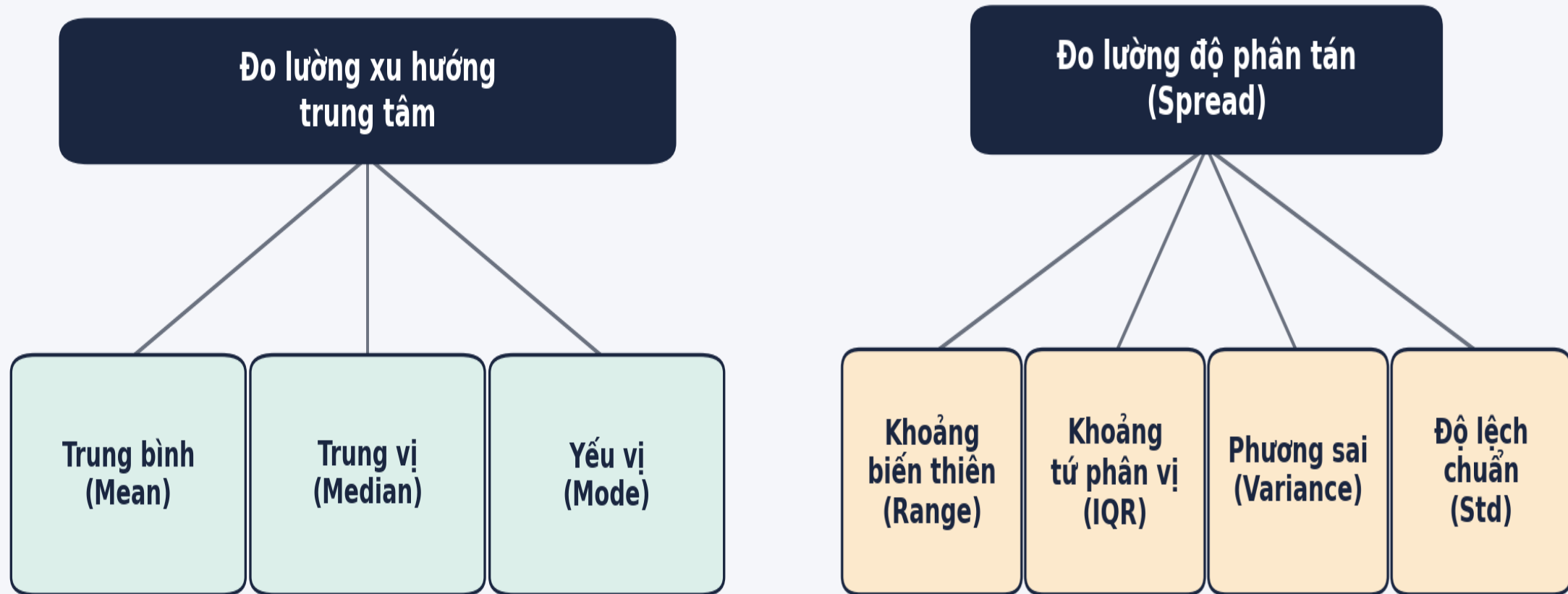
- Theo hạn ngạch (Quota)
- Theo phán đoán (Judgment)
- Thuận tiện (Convenience)

# Phân loại Thống kê

- **Thống kê mô tả vs Thống kê suy diễn:**
- **Thống kê mô tả (Descriptive):** tóm tắt, mô tả dữ liệu.
  - Dữ liệu số: trung tâm, độ biến thiên và hình dạng.
  - Dữ liệu phân loại: tần suất, phần trăm hoặc tỷ lệ.
- **Thống kê suy diễn (Inferential):** rút ra kết luận vượt ra ngoài mẫu đã nghiên cứu (dựa trên xác suất), cho phép dự đoán.

# Thống kê mô tả

Chia thành 2 nhóm chính:



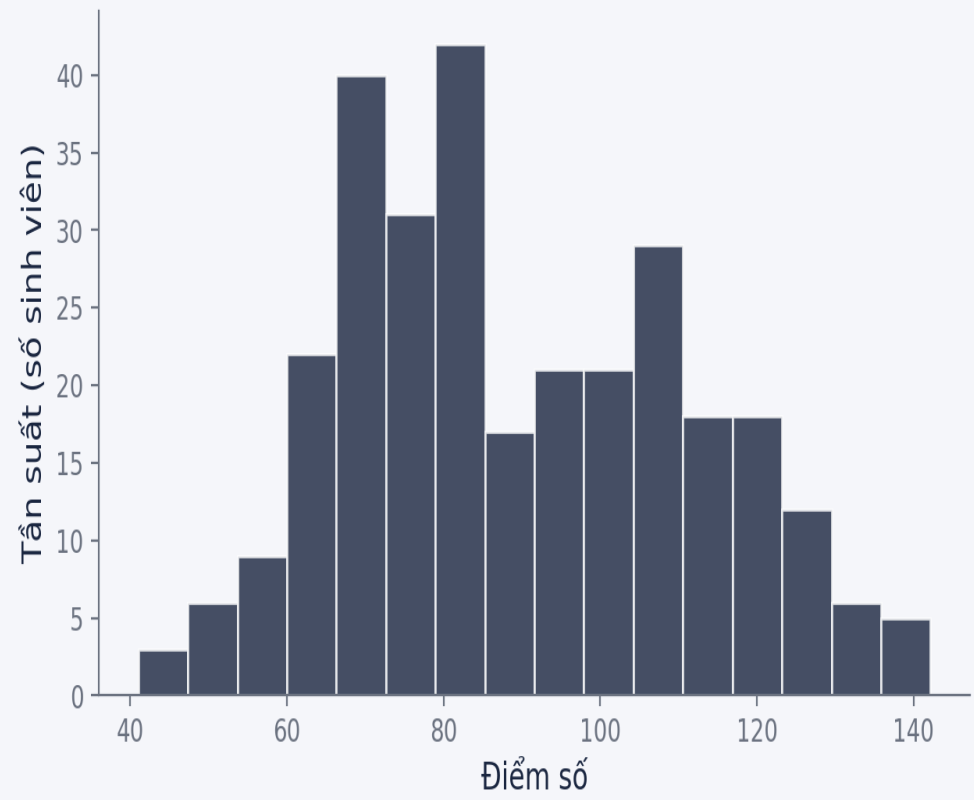
# Mô tả dữ liệu bằng thống kê

- **Thống kê mô tả một tập dữ liệu số bằng:**
  - Trung tâm (trung bình, trung vị, yếu vị...)
  - Độ biến thiên (độ lệch chuẩn, khoảng biến thiên...)
  - Hình dạng (độ lệch – skewness, độ nhọn – kurtosis...)
- **Thống kê mô tả một tập dữ liệu phân loại bằng:**
  - Tần suất, phần trăm hoặc tỷ lệ của mỗi nhóm.

# Thống kê giúp ích cho ta như thế nào

Chỉ nhìn vào dữ liệu thô, ta khó đưa ra mô tả hữu ích; nhưng thống kê cho ta cái nhìn nhanh và sâu vào dữ liệu thông qua các công cụ đồ họa và số liệu.

Phân bố điểm thi



| Thống kê                  | Giá trị  |
|---------------------------|----------|
| Trung bình (Mean)         | 90.42    |
| Trung vị (Median)         | 84       |
| Yếu vị (Mode)             | 84       |
| Độ lệch chuẩn (Std)       | 30.23    |
| Phương sai (Variance)     | 913.84   |
| Độ lệch (Skewness)        | 0.39     |
| Khoảng biến thiên (Range) | 95       |
| Min / Max                 | 48 / 143 |
| Số lượng (Count)          | 60       |

# Từ vựng Thống kê

- **Tổng thể (Population):** toàn bộ nhóm cần lấy thông tin.
- **Mẫu (Sample):** tập con của tổng thể; đặc trưng được xem là đại diện cho tổng thể.
- **Biến (Variable):** các đại lượng được đo trong mẫu.
  - **Định lượng (Quantitative) — số:**
    - Liên tục (ví dụ: độ pH, mức cholesterol).
    - Rời rạc (ví dụ: số khuẩn lạc trong môi trường nuôi cấy).
  - **Phân loại (Categorical):**
    - Định danh (Nominal — ví dụ: giới tính, nhóm máu).
    - Thứ bậc (Ordinal — ví dụ: bệnh nhẹ / vừa / nặng); thường được mã hóa lại thành biến định lượng.

# Hiệp phương sai (Covariance)

Hiệp phương sai đo lường xu hướng hai biến biến thiên cùng nhau.

Công thức tổng thể:

$$Cov(X, Y) = \frac{1}{N} \sum_{i=1}^N (x_i - \mu_x)(y_i - \mu_y)$$

Công thức mẫu:

$$Cov(X, Y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

**Cov > 0** → hai biến tăng/giảm cùng chiều; **Cov < 0** → ngược chiều; **Cov = 0** → không liên hệ tuyến tính.

## Ví dụ: Tính Hiệp phương sai

```
# Dữ liệu: Số giờ học (X) vs Điểm thi (Y)
X = [2, 4, 6, 8]      # số giờ học
Y = [50, 65, 75, 90]  # điểm thi
import numpy as np
cov_matrix = np.cov(X, Y)
print(cov_matrix[0,1]) # Cov(X,Y) = +26.67 (dương!)
```

### Diễn giải:

- **$\text{Cov}(X,Y) > 0$**   $\rightarrow$  học nhiều giờ hơn  $\rightarrow$  điểm cao hơn (cùng chiều).
- **$\text{Cov}(X,Z) < 0$**   $\rightarrow$  học nhiều hơn  $\rightarrow$  ít thời gian chơi game hơn (ngược chiều).
- **$\text{Cov} = 0$**   $\rightarrow$  hai biến không liên hệ (không tương quan).
- **Ứng dụng ML (Kỹ thuật đặc trưng):** | hiệp phương sai | lớn giữa hai đặc trưng = đa cộng tuyến. Bỏ bớt một đặc trưng để mô hình ổn định và dễ diễn giải hơn.

# Tương quan (Correlation)

- Hệ số tương quan không có đơn vị và luôn nằm trong khoảng từ  $-1$  (phản tương quan hoàn hảo) đến  $+1$  (tương quan hoàn hảo). Giá trị như  $0.25$  thể hiện tương quan dương khá yếu.
- Được định nghĩa bởi:

$$\rho_{X,Y} = \frac{Cov(X, Y)}{\sigma_X \sigma_Y}$$

$\rho$  = Hiệp phương sai chuẩn hóa theo độ lệch chuẩn của hai biến

# Ví dụ: Tương quan Pearson với Python

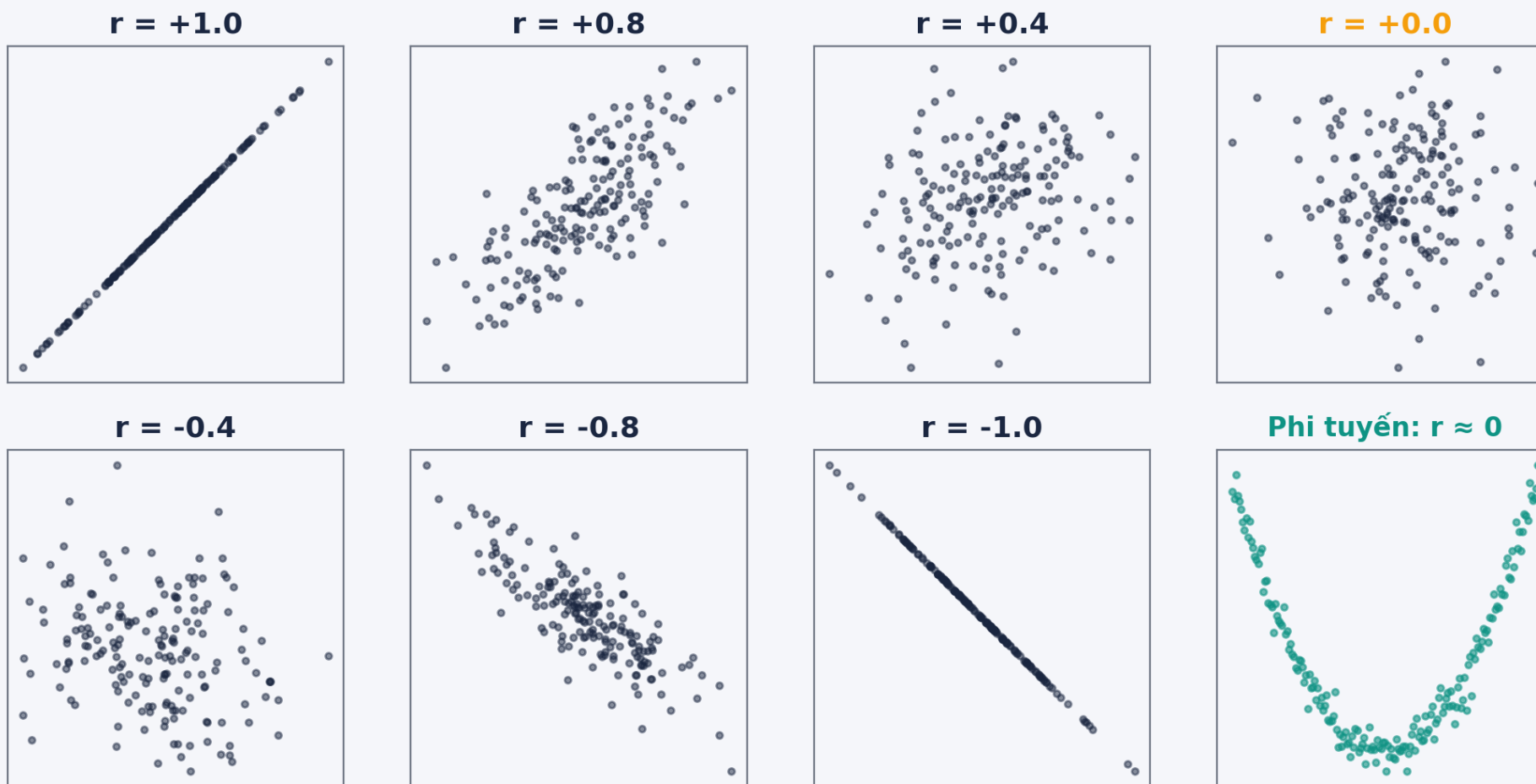
```
# Tương quan giữa số giờ học và điểm thi:
import scipy.stats as stats
import numpy as np
hours = np.array([2, 4, 6, 8, 10])
scores = np.array([50, 65, 75, 88, 95])
r, p_value = stats.pearsonr(hours, scores)
print(f'r = {r:.3f}')          # r = 0.993
print(f'p = {p_value:.4f}')    # p = 0.0007 (có ý nghĩa thống kê)
```

## Diễn giải hệ số r:

- 0.9 – 1.0 : tương quan dương rất mạnh
- 0.7 – 0.9 : tương quan dương mạnh
- 0.5 – 0.7 : tương quan dương trung bình
- Dưới 0.5 : yếu (cần xét yếu tố khác)
- $p\text{-value} < 0.05$  xác nhận tương quan là thật, không do ngẫu nhiên.

# Ví dụ minh họa Tương quan

Hệ số tương quan  $r$ : từ tương quan dương hoàn hảo (+1) đến âm hoàn hảo (-1)



05

# Entropy & Lý thuyết thông tin

*Entropy & Information Theory*

# Entropy

Entropy đo lường mức độ bất định / không thuần khiết (impurity) trong dữ liệu.

$$H(X) = - \sum_{i=1}^n p_i \log_2 p_i$$

trong đó:

- **X** – tập tất cả các mẫu trong dữ liệu
- **n** – số lớp (giá trị) phân biệt
- **p<sub>i</sub>** – xác suất của sự kiện thứ i

# Entropy: Ví dụ câu hỏi trắc nghiệm

**Công thức:**  $H = -\sum p_i \cdot \log_2(p_i)$  với  $p_i$  = xác suất của mỗi lựa chọn

*Câu hỏi 4 đáp án (A, B, C, D) — sinh viên chắc chắn đến đâu?*

```
# TH1 — Sinh viên biết đáp án (rất chắc chắn)
P(A)=0.97, P(B)=0.01, P(C)=0.01, P(D)=0.01
H = -(0.97·log2(0.97) + 3×0.01·log2(0.01)) ≈ 0.24 bit    # gần 0
```

```
# TH2 — Loại được 2 đáp án sai (còn chút bất định)
P(A)=0.5, P(B)=0.5, P(C)=0, P(D)=0
H = -(2 × 0.5·log2(0.5)) = 1.0 bit    # 50/50 giữa 2 lựa chọn
```

```
# TH3 — Đoán hoàn toàn ngẫu nhiên (bất định tối đa)
P(A)=P(B)=P(C)=P(D)=0.25
H = -(4 × 0.25·log2(0.25)) = 2.0 bit    # tối đa = log2(4)
```

# Entropy: Tổng kết & Điểm chính

So sánh ba tình huống:

| Tình huống        | Phân phối                | Entropy                   |
|-------------------|--------------------------|---------------------------|
| Biết đáp án       | [0.97, 0.01, 0.01, 0.01] | $\approx 0.24$ bit (thấp) |
| Loại 2 đáp án sai | [0.50, 0.50, 0.00, 0.00] | $= 1.00$ bit              |
| Đoán ngẫu nhiên   | [0.25, 0.25, 0.25, 0.25] | $= 2.00$ bit (max)        |

- **Điểm chính:**

- **Entropy = 0**  $\rightarrow$  hoàn toàn chắc chắn (chỉ một khả năng).
- **Entropy = max**  $\rightarrow$  hoàn toàn bất định (mọi khả năng như nhau).
- **Max với n lựa chọn =  $\log_2(n)$**  (ví dụ 4 lựa chọn  $\rightarrow$  max = 2.0 bit).
- **Trực giác:** entropy cao = cần thêm thông tin để quyết định; entropy thấp = đã đủ thông tin, ít bất ngờ.

# Ví dụ: Entropy trong Cây quyết định

Phân loại email: thư rác (S) vs không phải thư rác (NS)

```
# Nút gốc: 10 Spam, 10 Not-Spam
p_spam = 0.5, p_not = 0.5
H(root) = -(0.5*log2(0.5) + 0.5*log2(0.5)) = 1.0 bit
# Bất định tối đa (chia 50/50)

# Sau khi tách theo từ khóa 'discount':
Trái (có 'discount'): 8 Spam, 2 Not → H = 0.722 bit
Phải (không 'discount'): 2 Spam, 8 Not → H = 0.722 bit
H trung bình có trọng số = 0.722 bit
Độ lợi thông tin (Information Gain) = 1.0 - 0.722 = 0.278 bit
# Độ lợi cao = phép tách tốt! Chọn đặc trưng này.
```

Cây quyết định chọn đặc trưng có Độ lợi thông tin cao nhất tại mỗi nút.

06

# Kiểm định giả thuyết

*Hypothesis Testing*

# Kiểm định giả thuyết

- **Giả thuyết:** một phát biểu có thể kiểm chứng, ví dụ “đồng xu bị lệch”, “các nhóm khác nhau”.
- Ta muốn chứng minh giả thuyết  $H_1$ , nhưng việc đó khó, nên ta tìm cách bác bỏ giả thuyết không  $H_0$ .
- **Giả thuyết đối  $H_1$ :** ý tưởng của ta (ví dụ: đồng xu bị lệch, các nhóm khác nhau).
- **Giả thuyết không  $H_0$ :** trái với ý tưởng của ta (ví dụ: đồng xu cân bằng, các nhóm giống hệt nhau).
- **p-value:** ngưỡng xác suất; trực tiếp kiểm soát tỷ lệ dương tính giả (quan sát thấy chênh lệch lớn ngay cả khi  $H_0$  đúng).
- **Mức ý nghĩa ( $\alpha$ ):** giá trị p cho trước; thường dùng 0.05, 0.02, 0.01, 0.005, 0.001.
- **Nếu  $p\text{-value} < \alpha \rightarrow$  chấp nhận  $H_1$  (bác bỏ  $H_0$ ).**

# Ví dụ: Kiểm định giả thuyết cho mô hình ML (A/B Test)

Bối cảnh: so sánh độ chính xác của Mô hình A vs Mô hình B

- $H_0$  (không):  $\text{Accuracy}(A) = \text{Accuracy}(B)$
- $H_1$  (đối):  $\text{Accuracy}(A) \neq \text{Accuracy}(B)$

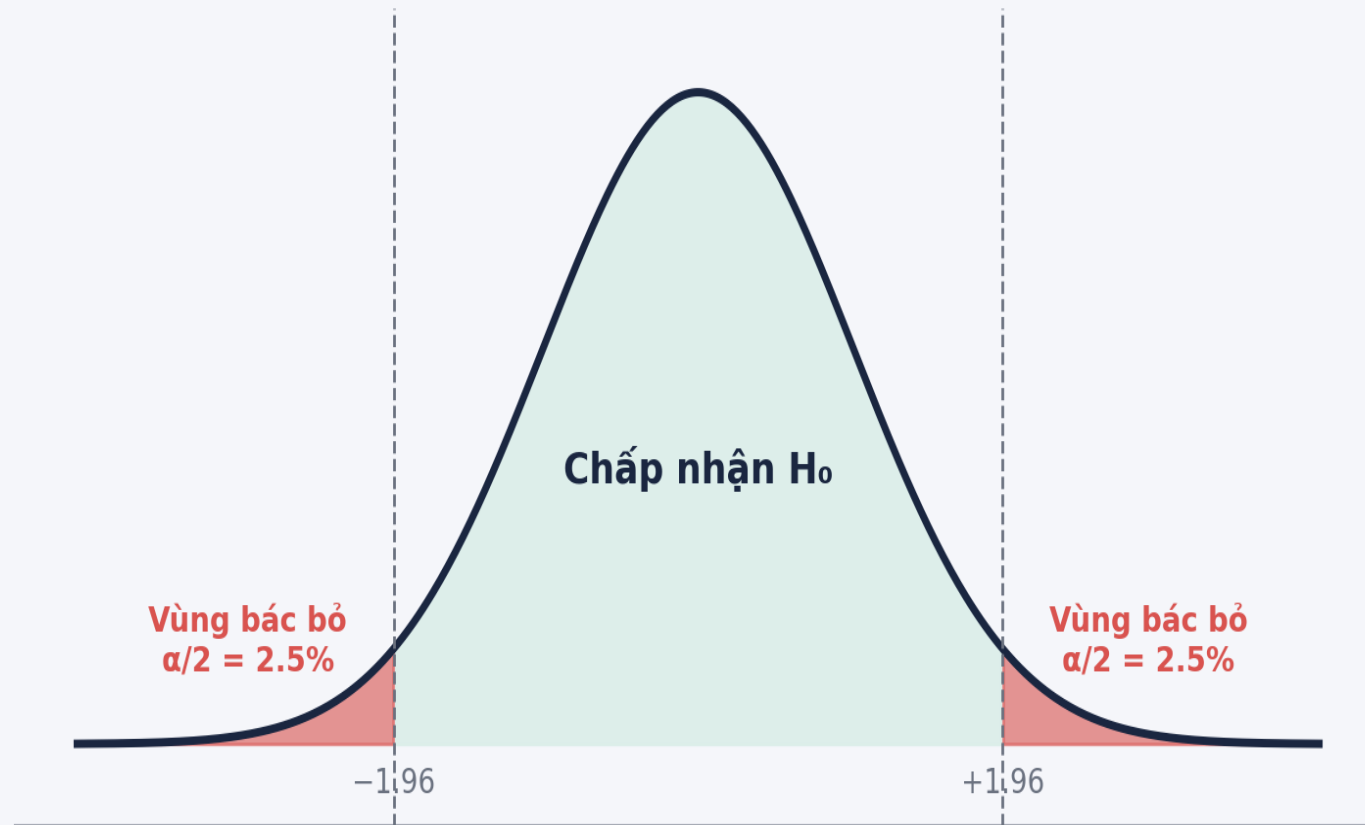
```
# Kết quả thử nghiệm trên 1000 mẫu kiểm thử:  
from scipy import stats  
# Mô hình A: đúng=850, n=1000 → 85.0% chính xác  
# Mô hình B: đúng=820, n=1000 → 82.0% chính xác  
t_stat, p_value = stats.ttest_ind(results_A, results_B)  
print(f'p-value = {p_value:.3f}') # p = 0.031
```

**Quyết định:**  $p = 0.031 < 0.05 = \alpha \rightarrow$  Bác bỏ  $H_0$ . Kết luận: Mô hình A tốt hơn Mô hình B một cách có ý nghĩa.

*Nếu  $p \geq 0.05$ : chưa thể kết luận có khác biệt (có thể cần thêm dữ liệu).*

# Kiểm định giả thuyết: Hai phía

Kiểm định hai phía (Two-tailed), mức ý nghĩa  $\alpha = 5\%$



Khi p-value nhỏ hơn 5% ( $p < .05$ ), ta bác bỏ giả thuyết không  $H_0$ .

# Kiểm định giả thuyết: Các loại kiểm định

- Kiểm định tương quan Pearson (Pearson correlation test)
- Kiểm định F (ANOVA) – F-Test
- Kiểm định T (T-Test):
  - Một mẫu (One sample)
  - Hai mẫu (Two sample)

# Kiểm định tương quan Pearson

- Kiểm tra mối liên hệ giữa hai biến định lượng. Phép kiểm tính hệ số tương quan Pearson và p-value để kiểm định giả thuyết không-tương-quan.
- p-value là xác suất quan sát được giá trị thống kê t cách 0 ít nhất bằng mức đó, nếu giả thuyết không (“X và Y độc lập”) là đúng.
- **Ví dụ:** nếu X và Y tương quan hoàn hảo thì p-value rất nhỏ ( $\approx 0$ ).

```
import scipy.stats as stats  
correlation, pvalue = stats.pearsonr(x, y)  # đúng: pearsonr
```

# Ví dụ: Kiểm định tương quan Pearson

```
import scipy.stats as stats
import numpy as np
# Dữ liệu: số giờ học vs điểm thi
X = np.array([2, 4, 6, 8, 10])
Y = np.array([50, 65, 75, 88, 95])
# Tên hàm đúng: pearsonr
correlation, p_value = stats.pearsonr(X, Y)
print(f'r          = {correlation:.3f}') # 0.993 → rất mạnh
print(f'p-value = {p_value:.4f}')      # 0.0007 → có ý nghĩa
# Diễn giải
if p_value < 0.05:
    print('Bác bỏ H0: X và Y tương quan có ý nghĩa thống kê')
```

# Kiểm định F (F-Test)

- Kiểm tra xem 2 (hoặc nhiều) phương sai của 2 (hoặc nhiều) tập dữ liệu có bằng nhau hay không.
- Dùng p-value để chấp nhận / bác bỏ giả thuyết.

```
import scipy.stats as stats  
f_val, p_val = stats.f_oneway(x, y, z)
```

# Kiểm định T (T-Test)

- **Một mẫu (One sample):** kiểm tra trung bình của tập dữ liệu có bằng ( $H_0$ ) một giá trị trung bình cho trước hay không.
- **Hai mẫu (Two sample):** kiểm tra trung bình của hai tổng thể có bằng/giống nhau hay không.

```
import scipy.stats as stats
# Một mẫu
t_stats, p_val = stats.ttest_1samp(X, given_mean)
# Hai mẫu
T_stats, p_val = stats.ttest_ind(x, y)
```

# Thống kê suy diễn

**Ước lượng tham số (Parameter Estimation):** dùng dữ liệu mẫu để tính một giá trị xấp xỉ tốt nhất cho tham số chưa biết của tổng thể.



⚠ Mẫu chỉ lấy từ MỘT vùng → KHÔNG đại diện cho cả châu Á (thiên lệch lấy mẫu / sampling bias)

# Ước lượng tham số

- **Phương pháp mô-men (Method of Moments):** ước lượng bằng cách cho  $k$  mô-men mẫu đầu tiên bằng  $k$  mô-men tổng thể tương ứng.
- **Ước lượng hợp lý cực đại (MLE):** dùng một mô hình và các giá trị trong mô hình để cực đại hóa hàm hợp lý (likelihood).
- **Ước lượng Bayes – Cực đại hậu nghiệm (MAP):** cực đại hóa kỳ vọng của biến ngẫu nhiên.
- **Ước lượng không chệch tốt nhất (best unbiased estimator).**

07

# Tối ưu hóa

*Optimization*

# Tối ưu hóa: Hàm chi phí

- **Hàm chi phí (Cost Function):** hàm toán học dùng để đánh giá mô hình hoạt động tốt đến đâu.
- **Ví dụ:**
  - **Sai số toàn phương trung bình (MSE)** cho bài toán hồi quy.
  - **Entropy chéo (Cross-Entropy)** cho bài toán phân loại.
- **Mục tiêu:** cực tiểu hóa hàm chi phí để cải thiện hiệu năng mô hình.

# Ví dụ: Tính MSE và Cross-Entropy

Sai số toàn phương trung bình (Hồi quy):

```
y_true = [3.0, 5.0, 7.0]
y_pred = [2.8, 5.2, 6.5]
errors = [(3.0-2.8)^2, (5.0-5.2)^2, (7.0-6.5)^2]
        = [0.04,          0.04,          0.25]
MSE     = (0.04+0.04+0.25)/3 = 0.11   # nhỏ = mô hình tốt
```

Entropy chéo nhị phân (Phân loại):

```
# y_true=1 (mèo), y_pred=0.9   (mô hình tự tin → đúng)
L = -(1 * log(0.9)) = 0.105      # mất mát nhỏ
# y_true=1 (mèo), y_pred=0.1   (mô hình sai → vẫn tự tin)
L = -(1 * log(0.1)) = 2.303      # mất mát lớn!
```

**Cross-Entropy phạt rất nặng các dự đoán sai mà lại tự tin → buộc mô hình vừa chính xác vừa hiệu chỉnh tốt độ tin cậy.**

# Tối ưu hóa: Tối ưu lồi

- **Hàm lồi (Convex Function):** hàm mà đoạn thẳng nối hai điểm bất kỳ trên đồ thị luôn nằm phía trên đồ thị.
- **Ý nghĩa:** nhiều mô hình học máy (như SVM) dựa vào tối ưu lồi để tìm tham số tốt nhất.
- **Ứng dụng:** tối ưu lồi đảm bảo cực tiểu cục bộ chính là cực tiểu toàn cục, giúp đơn giản hóa quá trình tối ưu.

# Tối ưu hóa: Chính quy hóa (Regularization)

- **Quá khớp & Dưới khớp:** quá khớp (overfitting) xảy ra khi mô hình quá phức tạp; dưới khớp (underfitting) khi mô hình quá đơn giản.
- **Các kỹ thuật chính quy hóa:**
  - **L1 (Lasso):** cộng giá trị tuyệt đối của trọng số vào hàm chi phí.
  - **L2 (Ridge):** cộng bình phương trọng số vào hàm chi phí.
- **Mục tiêu:** khuyến khích mô hình đơn giản hơn, tổng quát hóa tốt hơn trên dữ liệu chưa thấy.

# Ví dụ: Chính quy hóa L1 vs L2 trong Python

## L1 (Lasso) — ép trọng số thừa (nhiều giá trị 0):

```
from sklearn.linear_model import Lasso, Ridge
lasso = Lasso(alpha=0.1); lasso.fit(X_train, y_train)
print(lasso.coef_)
# [1.2, 0.0, 0.0, 3.4, 0.0] <- nhiều số 0 = tự chọn đặc trưng
```

## L2 (Ridge) — co đều mọi trọng số:

```
ridge = Ridge(alpha=0.1); ridge.fit(X_train, y_train)
print(ridge.coef_)
# [1.1, 0.2, 0.3, 3.2, 0.1] <- đều nhỏ nhưng khác 0
```

## So sánh:

- **L1 (Lasso):** Phạt =  $\sum |w|$ . Tốt khi nhiều đặc trưng không liên quan.
- **L2 (Ridge):** Phạt =  $\sum w^2$ . Tốt khi mọi đặc trưng đều đóng góp.
- **ElasticNet:** kết hợp cả L1 + L2 để điều khiển linh hoạt.